

AI 駆動開発ドキュメント PDF 化検証

Markdown で作成した開発ドキュメントを Vivliostyle で提出用 PDF に変換するための検証サンプル。

項目	内容
作成日	2026-06-30
作成者	開発チーム
版数	0.1
文書種別	検証用サンプル

変更履歴

版数	日付	変更内容	作成者
0.1	2026-06-30	初版作成。Markdown、表、コード、Mermaid、脚注、CSS の検証項目を追加。	開発チーム

目次

1	はじめに	4
1.1	基本表現	4
2	AI 駆動開発の作業計画	5
2.1	作業フェーズ	5
2.2	長い表の確認	5
3	設計方針	6
3.1	文書テンプレートの考え方	6
3.1.1	注意書き	6
3.1.2	補足	6
4	サンプルコード	7
4.1	TypeScript	7
4.2	横に長いコード	7
4.3	JSON	7
5	Mermaid 図表サンプル	8
5.1	Markdown 内に直接書く Mermaid	8
5.2	Mermaid から変換した SVG	8
6	テスト計画	9
7	リスクと対策	10
8	補足資料	11
8.1	検証メモ	11

1 はじめに

この文書は、AI駆動開発で日常的に作成する Markdown ドキュメントを、提出用 PDF として整形できるかを確認するためのサンプルです。要件整理、設計方針、実装計画、テスト計画、リスク管理を一つの文書に含め、Vivliostyle の基本的な組版機能を確認します。

日 本 語 の 文 章 、 English words 、 `inline code` 、 URL <https://example.com/really/long/path/that/should/wrap/properly/in/pdf/output> を含め、改行や折り返しも確認します。

AI を使う開発では、作業の過程そのものが重要な判断記録になります。Markdown を一次情報として残し、提出物として PDF 化できることは、レビューと共有の両方に役立ちます。

1.1 基本表現

- 箇条書きの第1階層
 - ネストした項目
 - 追加の補足
- 太字、斜体、`inline code`
- [内部リンク: Mermaid 図表サンプル](#)

1. 調査する
 2. 設計する
 3. 実装する
 4. 検証する
-

2 AI 駆動開発の作業計画

2.1 作業フェーズ

フェーズ	目的	成果物	確認観点
企画	解くべき問題を明確にする	課題メモ、成功条件	目的が曖昧でないか
要件整理	必要な機能と制約を整理する	要件定義書	スコープが適切か
設計	実装前に構成を固める	設計書、API 仕様	レビュー可能か
実装	AI と協働してコードを作る	Pull Request	差分が追えるか
検証	品質を確認する	テスト結果、PDF 文書	再現性があるか

2.2 長い表の確認

ID	タスク	担当	優先度	ステータス	備考
T-001	Markdown から PDF を生成する最小構成を作る	Dev	High	Doing	Vivliostyle CLI の設定、CSS、出力先を確認する
T-002	Mermaid 図を直接記述した場合の見え方を確認する	Dev	High	Todo	直接レンダリングできない場合は、事前変換した SVG または PNG を利用する
T-003	横に長いコードブロックと URL の折り返しを確認する	Dev	Medium	Todo	提出用 PDF では読みやすさを優先し、必要に応じてフォントサイズを調整する

3 設計方針

3.1 文書テンプレートの考え方

提出用 PDF では、派手な装飾よりも、読みやすさ、章立ての明確さ、表とコードの安定した表示を優先します。Markdown の記述量を増やしすぎず、CSS 側でできる限り体裁を整えます。

3.1.1 注意書き

Vivliostyle の Markdown 処理だけでは Mermaid が自動描画されない可能性があります。その場合は Mermaid CLI で SVG に変換し、画像として貼り込む運用を検証します。

3.1.2 補足

脚注の表示も確認します。AI が生成した設計案を採用する場合は、判断理由と検証結果を残すことが重要です。[1](#)

4 サンプルコード

4.1 TypeScript

```
type TaskStatus = 'todo' | 'doing' | 'done';

type DevelopmentTask = {
  id: string;
  title: string;
  status: TaskStatus;
  owner: string;
};

const tasks: DevelopmentTask[] = [
  { id: 'T-001', title: 'Create Vivliostyle config', status: 'doing', owner: 'Dev' },
  { id: 'T-002', title: 'Render Mermaid diagrams', status: 'todo', owner: 'Dev' },
];

const activeTasks = tasks.filter((task) => task.status !== 'done');
console.log(activeTasks);
```

4.2 横に長いコード

```
npm run build -- --output dist/vivliostyle-sample.pdf --theme styles/document.css --size A4 --timeout 120000
```

4.3 JSON

```
{
  "documentType": "development-plan",
  "output": "pdf",
  "checks": ["markdown", "tables", "code", "images", "mermaid", "footnotes", "paged-media"]
}
```

5 Mermaid 図表サンプル

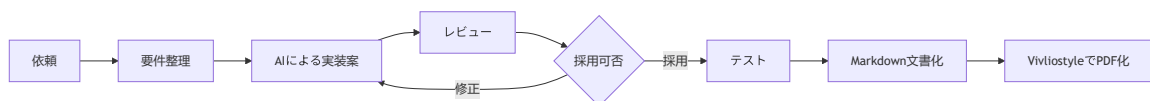
5.1 Markdown 内に直接書く Mermaid

```
flowchart TD
    A[要件を整理] --> B[AIに実装案を生成させる]
    B --> C{レビュー結果}
    C -->|問題なし| D[実装を採用]
    C -->|修正あり| E[プロンプトと設計を調整]
    E --> B
    D --> F[テストとPDF化]
```

```
sequenceDiagram
    participant User as 利用者
    participant AI as AIエージェント
    participant Repo as リポジトリ
    User->>AI: 要件と制約を伝える
    AI->>Repo: 実装と文書を更新する
    Repo-->>AI: テスト結果を返す
    AI-->>User: 変更内容とPDFを共有する
```

5.2 Mermaid から変換した SVG

次の図は `docs/diagrams/ai-driven-workflow.mmd` を Mermaid CLI で SVG に変換して貼り込む想定です。



AI 駆動開発ワークフロー

6 テスト計画

テスト対象	確認内容	期待結果
ページ番号	本文ページに番号が表示される	フッターにページ番号が表示される
コード	長い行が枠外にはみ出さない	折り返し、または読みやすい幅に収まる
表	セル内の長文が折り返される	ページ幅内に収まる
Mermaid	直接記述と画像貼り込みを比較する	どちらの運用が安定するか判断できる
脚注	ページ下部に表示される	本文と重ならない

7 リスクと対策

リスク	影響	対策
Mermaid が直接描画されない	図がコードブロックとして表示される	SVG または PNG に事前変換する
日本語フォントが環境差で変わる	PDF の見た目が安定しない	フォント指定を CSS に集約する
横長の表やコードがはみ出す	提出物として読みにくい	折り返し、フォントサイズ、表設計を調整する
ページ分割が不自然になる	章や図表が読みづらい	<code>break-before</code> 、 <code>break-inside</code> を調整する

8 補足資料

8.1 検証メモ

このサンプルは、PDF 出力の見た目を確認しながら改善していく前提です。最初の PDF では Mermaid の直接描画、表の幅、コードブロックの改ページを重点的に確認します。

1. 判断ログは、あとから仕様変更や不具合調査を行うときの根拠になります。 [🔗](#)